Homework 08

Author: Manushri Pendekanti (mp79)

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
```

Question 1: Boundary and margin

Use `data/margin-geometry-20.csv` (20 observations, predictors x_1, x_2, labels $y \in \{-1, 1\}$).

Score: $f(x) = 0.8 x_1 + 0.6 x_2$. The lines $f(x) = \pm 1$ are each distance 1 from the boundary $f(x) = 0$.

Do not fit a new classifier for this question.

In [2]:
```
#setup

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

dat = pd.read_csv("data/margin-geometry-20.csv")

# Use the observation numbers from the file if available.
# If there is no observation-number column, use the row numbers.
id_candidates = ["observation", "obs", "id"]
id_column = next(
    (column for column in dat.columns
     if column.lower() in id_candidates),
    None
)

if id_column is None:
    dat["observation"] = np.arange(1, len(dat) + 1)
else:
    dat["observation"] = dat[id_column]

# Calculate score, distance, and functional margin
dat["score"] = 0.8 * dat["x1"] + 0.6 * dat["x2"]
dat["distance_to_boundary"] = dat["score"].abs()
dat["functional_margin"] = dat["y"] * dat["score"]

# Compare yf(x) with 1
tolerance = 1e-8
```

```
dat["margin_status"] = np.where(
    np.isclose(dat["functional_margin"], 1, atol=tolerance),
    "On margin",
    np.where(
        dat["functional_margin"] < 1,
        "Inside margin",
        "Beyond margin"
    )
)

dat["misclassified"] = dat["functional_margin"] < 0
```

Question 1 — Part a

(Your written answer.)

In [3]:
```
# Distance to boundary, functional margin y*f(x), inside/on/beyond margin

margin_table = dat[
    [
        "observation",
        "x1",
        "x2",
        "y",
        "distance_to_boundary",
        "functional_margin",
        "margin_status",
    ]
]

print(margin_table.to_string(index=False))
```

```
 observation    x1    x2  y  distance_to_boundary  functional_margin margin_
status
           1  0.68  0.76  1                  1.00               1.00     On
margin
           2  1.80 -0.15  1                  1.35               1.35 Beyond
margin
           3  1.38  0.66  1                  1.50               1.50 Beyond
margin
           4  0.88  1.66  1                  1.70               1.70 Beyond
margin
           5  0.62  2.34  1                  1.90               1.90 Beyond
margin
           6  2.78 -0.04  1                  2.20               2.20 Beyond
margin
           7  1.86  1.52  1                  2.40               2.40 Beyond
margin
           8  0.50  0.40  1                  0.64               0.64 Inside
margin
           9  0.10  0.20  1                  0.20               0.20 Inside
margin
          10 -0.40  0.20  1                  0.20              -0.20 Inside
margin
          11 -0.92 -0.44 -1                  1.00               1.00     On
margin
          12 -0.50 -1.50 -1                  1.30               1.30 Beyond
margin
          13 -1.90 -0.05 -1                  1.55               1.55 Beyond
margin
          14 -0.44 -2.33 -1                  1.75               1.75 Beyond
margin
          15 -1.98 -0.61 -1                  1.95               1.95 Beyond
margin
          16 -2.74  0.07 -1                  2.15               2.15 Beyond
margin
          17 -1.90 -1.55 -1                  2.45               2.45 Beyond
margin
          18 -0.50 -0.40 -1                  0.64               0.64 Inside
margin
          19  0.00 -0.40 -1                  0.24               0.24 Inside
margin
          20  0.40 -0.20 -1                  0.20              -0.20 Inside
margin
```

Question 1 — Part b

(Your written answer.)

In [4]:

```
# Plot observations, boundary f(x)=0, margin lines f(x)=±1

fig, ax = plt.subplots(figsize=(8, 7))


x1_line = np.linspace(-3.2, 3.2, 300)

ax.plot(
```

```
    x1_line,
    (-0.8 * x1_line) / 0.6,
    color="black",
    label="Boundary: f(x) = 0"
)

ax.plot(
    x1_line,
    (-0.8 * x1_line + 1) / 0.6,
    color="gray",
    linestyle="--",
    label="Margin: f(x) = 1"
)

ax.plot(
    x1_line,
    (-0.8 * x1_line - 1) / 0.6,
    color="gray",
    linestyle="--",
    label="Margin: f(x) = -1"
)


class_colors = {-1: "blue", 1: "darkgreen"}

for y_value, color in class_colors.items():
    class_data = dat[dat["y"] == y_value]
    ax.scatter(
        class_data["x1"],
        class_data["x2"],
        color=color,
        label=f"Class y = {y_value}",
        s=55,
        zorder=3
    )

# Circle observations inside the margin
inside = dat[dat["functional_margin"] < 1]
ax.scatter(
    inside["x1"],
    inside["x2"],
    facecolors="none",
    edgecolors="orange",
    linewidths=2,
    s=150,
    label="Inside margin",
    zorder=4
)

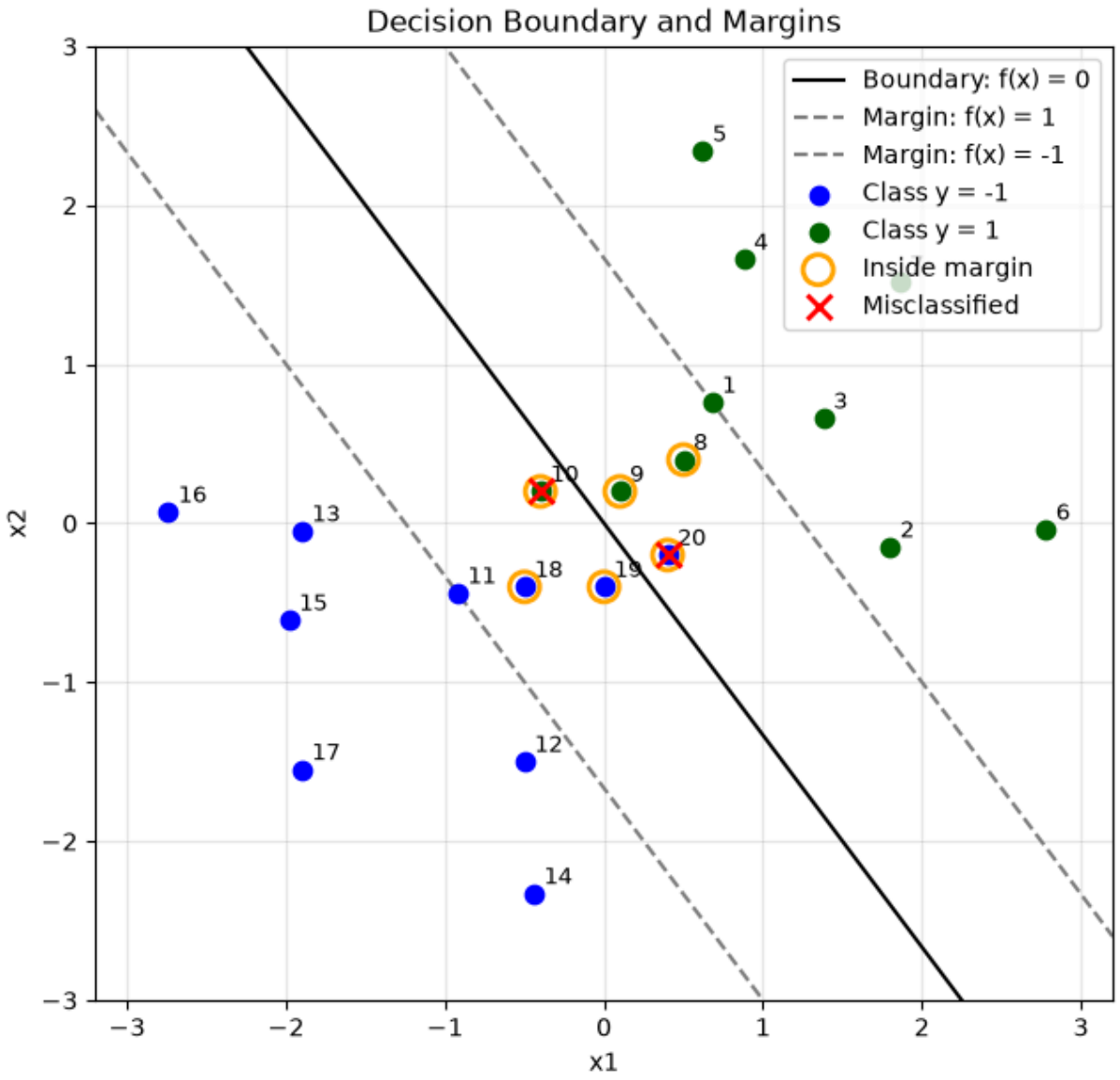

misclassified = dat[dat["misclassified"]]
ax.scatter(
    misclassified["x1"],
    misclassified["x2"],
    color="red",
    marker="x",
```

```
    linewidths=2,
    s=100,
    label="Misclassified",
    zorder=5
)


for _, row in dat.iterrows():
    ax.annotate(
        str(row["observation"]),
        (row["x1"], row["x2"]),
        xytext=(4, 4),
        textcoords="offset points",
        fontsize=9
    )

ax.set_xlim(-3.2, 3.2)
ax.set_ylim(-3, 3)
ax.set_aspect("equal", adjustable="box")
ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_title("Decision Boundary and Margins")
ax.legend()
ax.grid(True, alpha=0.3)

plt.show()
```



Question 1 — Part c

(Your written answer.)

In [5]:
```
# Count inside/on/beyond margin; list misclassified observations

status_order = ["Inside margin", "On margin", "Beyond margin"]

margin_counts = (
    dat["margin_status"]
    .value_counts()
    .reindex(status_order, fill_value=0)
)

print("Number of observations in each margin category:")
print(margin_counts.to_string())

misclassified_observations = dat.loc[
    dat["misclassified"], "observation"
```

```
].tolist()

print("\nMisclassified observations:")
if misclassified_observations:
    print(misclassified_observations)
else:
    print("None")
```

```
Number of observations in each margin category:
margin_status
Inside margin     6
On margin         2
Beyond margin    12

Misclassified observations:
[10, 20]
```

Question 2: Margin in four dimensions

Use `data/margin-geometry-4d.csv` (50 observations, four predictors, labels $y \in \{-1, 1\}$).

Score: $f(\mathbf{x}) = \beta_0 + \mathbf{x}^{\mathsf T}\boldsymbol{\beta}$ with $\beta_0 = -1$ and $\boldsymbol{\beta} = (1, 2, -2, 4)^{\mathsf T}$.

In [6]:
```
#setup

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

dat = pd.read_csv("data/margin-geometry-4d.csv")

predictor_columns = ["x1", "x2", "x3", "x4"]
beta = np.array([1, 2, -2, 4], dtype=float)
beta0 = -1

X = dat[predictor_columns].to_numpy(dtype=float)
y = dat["y"].to_numpy(dtype=float)

# Use the observation numbers from the data if available.
id_candidates = ["observation", "obs", "id"]
id_column = next(
    (column for column in dat.columns
     if column.lower() in id_candidates),
    None
)

if id_column is None:
    observation_numbers = np.arange(1, len(dat) + 1)
else:
    observation_numbers = dat[id_column].to_numpy()

beta_norm = np.linalg.norm(beta)
```

```
scores = beta0 + X @ beta
functional_margins = y * scores

# Project each observation onto the unit direction of beta.
projection = X @ (beta / beta_norm)

# In projection coordinates, the boundary is beta0 + ||beta|| * projection =
boundary_projection = -beta0 / beta_norm
```

Question 2 — Part a

(Your written answer.)

In [7]:
```
# Project onto direction of beta; plot; estimate geometric margin M

signed_distances = functional_margins / beta_norm
M = np.min(signed_distances)


absolute_distances = np.abs(projection - boundary_projection)
closest_index = np.argmin(absolute_distances)

print(f"Decision boundary on projection axis: {boundary_projection:.6f}")
print(f"Geometric margin M: {M:.6f}")
print(
    "Observation closest to the boundary:",
    observation_numbers[closest_index]
)
print(f"Absolute distance of that observation: {absolute_distances[closest_i

vertical_position = np.zeros(len(dat))
for class_value in [-1, 1]:
    indices = np.where(y == class_value)[0]
    vertical_position[indices] = np.linspace(-0.08, 0.08, len(indices))

colors = {-1: "blue", 1: "darkgreen"}

fig, ax = plt.subplots(figsize=(9, 3.5))

for class_value in [-1, 1]:
    indices = np.where(y == class_value)[0]
    ax.scatter(
        projection[indices],
        vertical_position[indices],
        color=colors[class_value],
        s=60,
        label=f"Class y = {class_value}"
    )


ax.axvline(
    boundary_projection,
    color="black",
    linestyle="--",
    label=f"Decision boundary = {boundary_projection:.3f}"
```

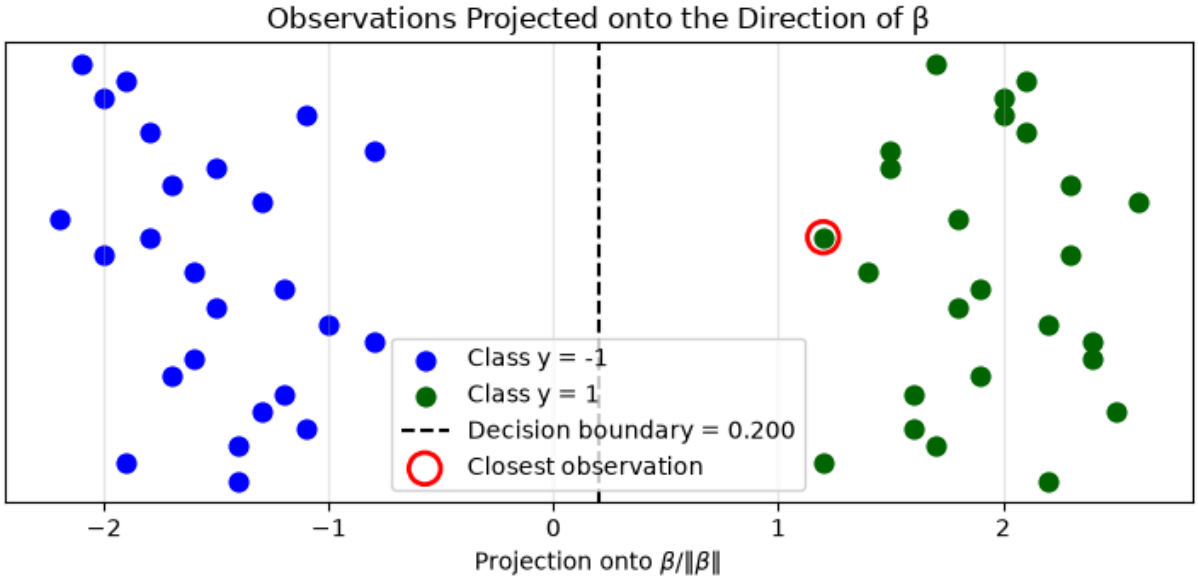
```
)


ax.scatter(
    projection[closest_index],
    vertical_position[closest_index],
    facecolors="none",
    edgecolors="red",
    s=180,
    linewidths=2,
    label="Closest observation"
)

ax.set_xlabel(r"Projection onto $\beta/\|\beta\|$")
ax.set_yticks([])
ax.set_title("Observations Projected onto the Direction of β")
ax.legend()
ax.grid(axis="x", alpha=0.3)

plt.show()
```

```
Decision boundary on projection axis: 0.200000
Geometric margin M: 1.000000
Observation closest to the boundary: 33
Absolute distance of that observation: 1.000000
```



Question 2 — Part b

(Your written answer.)

In [8]:
```
# Rescale to SVM normalization (smallest functional margin = 1)


minimum_functional_margin = np.min(functional_margins)

if minimum_functional_margin <= 0:
    print(
```

```
        "The smallest functional margin is not positive, so positive "
        "rescaling cannot make the smallest margin equal to 1."
    )
else:
    beta0_svm = beta0 / minimum_functional_margin
    beta_svm = beta / minimum_functional_margin

    print(f"Original smallest functional margin: {minimum_functional_margin:
    print(f"beta0 under SVM normalization: {beta0_svm:.6f}")
    print("beta under SVM normalization:", beta_svm)
```

```
Original smallest functional margin: 5.000000
beta0 under SVM normalization: -0.200000
beta under SVM normalization: [ 0.2  0.4 -0.4  0.8]
```

Question 3: Peer review of a Homework 7 skill

1. **Use the skill.** Copy the assigned student's skill into a local `SKILL.md`, then ask your agent to follow it on `data/saheart.csv` (Homework 7 coding for response/predictors).
2. **Write a peer review.** Brief overall assessment plus three major items the author could improve. Write the review yourself.
3. **Share your review.** Open a GitHub Issue in the assigned student's repo and include the same review plus a link to that Issue in this report.

Question 3

(Your written answer / skill path and invocation notes.)

Agent run on SAheart using peer SKILL.md (optional helper code)

Link to Issue Github:

https://github.com/yanxif2/stat432-fall2026/issues/4

Review: The report generated by the skill was very a lot of information it included the all the specfic values inlcuding detailed info about the setup, seed, etc and plots very detailed and easily understandable.

1. First thing I noticed was skill could have explained the findings and limitation with a little more detail while it has highlighted what important it didn't go into detail how they came to that conclusion and what specfic pieces of graph or results resulted in coming to that conclusion and why it matters because just reporting findings or

information without much background or explaination make it hard for a reader or someone trying to understand it with less background in this content or just someone trying use this results to make an important descision it hard for them to decide based on a findings without much explaination. What should be done is in the skill when the reporting or finding are reported make sure to add details on how those finding were decided and used data points as evidence

2. Second thing for the graphs while there are graphs related to PCA and QDA+LDA combined or blended I think there should other graphs related to what was shown in the table as well including just LDA, QDA, etc because the graphs give a lot of information and the reason why it matters because the graphs provide a lot of insight into the trends that are important that the table cannot provide when choosing a specfic model because table provides just important setup while graphs can help me see and compare trends and what should be done is when the skill is told me to come with graphs or create graphs it should also be told for what comparision it is making to include graph as supporting evidence

3. Finally for more readiability what I noticed was lot of information while it was all there it was not formatted well and it was all hard to follow and understand because of formatting because the sentences didn't use . but used ; instead and there was not a lot of spacing and why it matters because one important part of report when looking at is how formatted and how easy it follow if it hard to follow the whole purpose of report to give a quick understanding of why to pick a certain model and when report becomes hard to follow it loses the whole purpose. What I would change is it in the skill had more instructions on formatting and spacing and having information organizedd in a way it is easy to follow which would create a better report